RANDOMIZED ALGORITHMS

MODULE 6

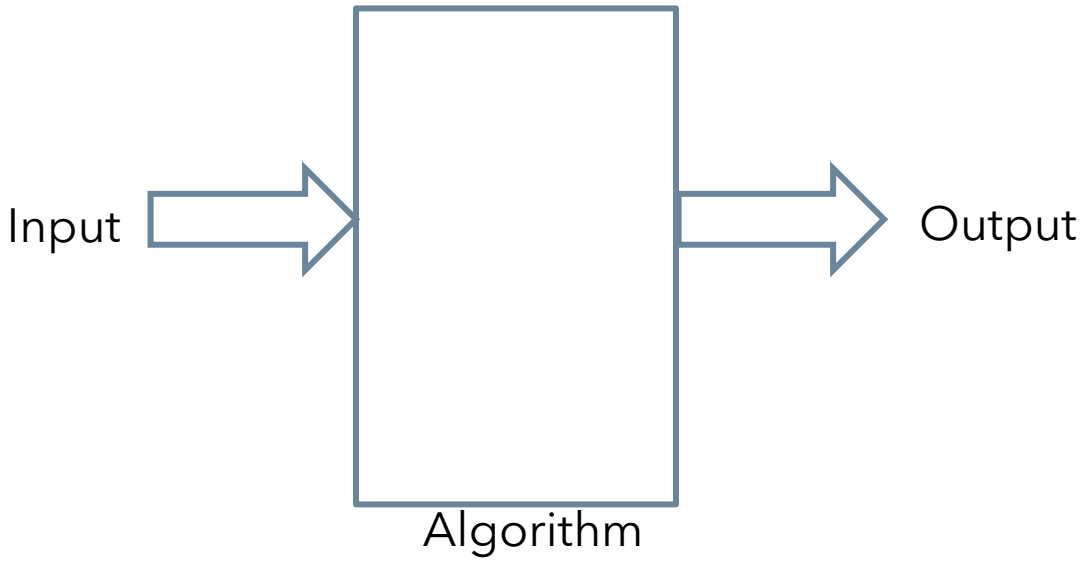
BCSE206L – DESIGN AD ANALYSIS OF ALGORITHMS

Randomized Algorithms

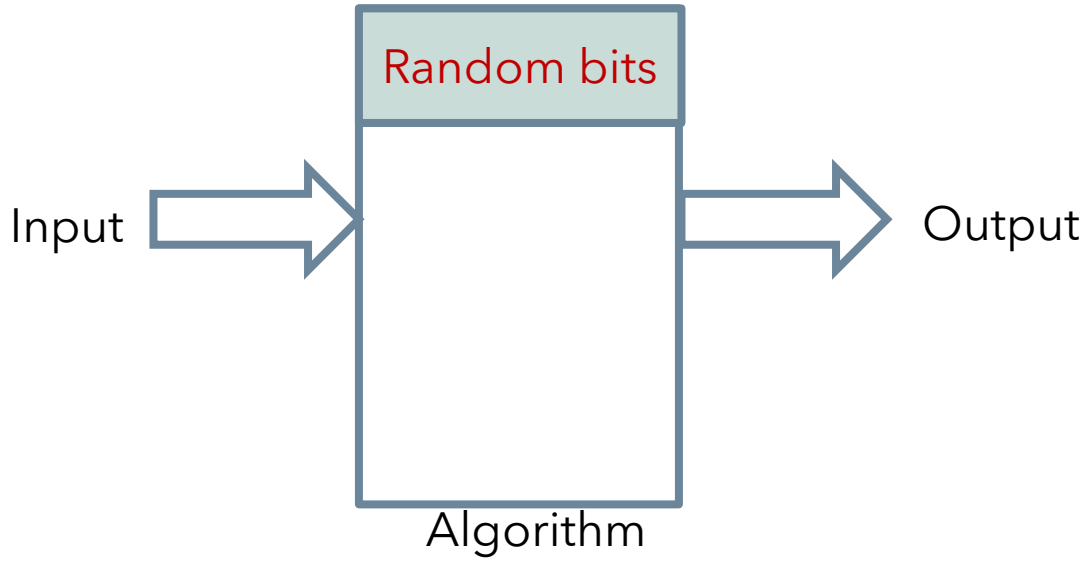
- Randomized algorithms **use a source of randomness to make their decisions**. They are used to reduce usage of resources such as time, space or memory by a standard algorithm.
- A standard, deterministic algorithm will always produce the same output given a particular input.
- It will always pass through the same steps to reach that output, i.e. its behavior does not change on different runs.
- A real life example of this would be a known chemical reaction. Two parts hydrogen and one part oxygen will always make two molecules of water.

Randomized Algorithms

- The output as well as the **running time** are **functions only of the input.**



- The output as well as the **running time** are **functions of the input and random bits chosen.**



Randomized Algorithms

- Advantage of Randomized Algorithm
- Paradigm: Instead of making a **guaranteed good choice**, make a **random choice** and hope that it is good. This help because guaranteeing a good choice become difficult sometimes.
- It make random choices. **The expected running time depends on the random choice**, **not on any input distribution**.
- **Average case analysis** – analyze the expected running time of deterministic algorithms assuming a suitable random distribution on the input.

Randomized Algorithms

- **Pros:**
- Making a random choice is **fast**.
- An adversary is powerless; randomized algorithms have **no worst case inputs.**
- Randomized algorithms are often **simpler and faster than their deterministic counterparts.**

Randomized Algorithms

- **Cons:**
- In the worst case, a randomized algorithm may be **very slow**.
- There is a **finite probability of getting incorrect answer**. However, the probability of getting a wrong answer can be made arbitrarily small by the repeated employment of randomness.
- Getting **true random number is almost impossible**.

Randomized Algorithms

- **Definition**
- **Las Vegas:** a randomized algorithm that always returns a correct result. But the running time may vary between executions.
  - Example: Randomized QUICKSORT Algorithm
- **Monte Carlo:** a randomized algorithm that terminates in polynomial time, but might produce erroneous result.
  - Example: Randomized MINCUT Algorithm

Randomized Algorithms

- **Las Vegas Algorithm:**
- Output is always correct
- Running time is a random variable
- Example: Randomized Quick sort
- Probability [running time of quick sort exceed twice its expected time $< n^{loglogn}$
-

Randomized Algorithms

- **Monte Carlo Algorithm:**
- Output is incorrect with some probability
- Running time is deterministic
- Example: Randomized algorithm for approximate median
-

RANDOMIZED QUICK SORT

MODULE 6

Randomized Quick Sort

- **Problem:** Given an Array $A[]$ containing n elements, sort them in increasing/decreasing order
- **QSORT(A,P,q)**

  IF $p \geq q$, EXIT

    Compute s ← correct position of A[p] in the sorted order of the elements of A from p-th location to q-th location.

  Move the pivot A[p] into position A[s].

  Move the remaining elements of A[p..q] into appropriate sides.

  QSORT(A, p, s -1);

  QSORT(A, s + 1, q).

Randomized Quick Sort

- **Complexity:**
- The worst case time complexity is **$O(n^2)$**.
- The average case time complexity is **O(nlogn)**.

Randomized Quick Sort – Version 1

- The **Central Splitter**: It is an index s that the number of element less than $A[s]$ is at least **n/4**.
- The algorithm randomly chooses a key, and check whether it is a **central splitter** or not.
- If it is a **central splitter**, then the array is split with that key as was done in the QSORT algorithm
- It can be shown that the expected number of trials needed to get a **central splitter** is constant.

Randomized Quick Sort – Version 1

- RandQSORT(A, **p, q**)

If $p \geq q$, then EXIT.

While no **central splitter** has been found, execute the following steps:

Choose uniformly at random a number $r \in \{p, p+1, \ldots, q\}$.

Compute s = number of elements in A that are less than A[r], and t = number of elements in A that are greater than A[r].

If $\boldsymbol{s \geq (q-p)/4}$ and $\boldsymbol{t \geq (q-p)/4}$, then A[r] is a **central splitter.**

Position A[r] in A[s + 1], put the members in A that are smaller than the **central splitter** in A[p..s] and the members in A that are larger than the **central splitter** in A[s + 2…q].

RandQSORT(A, p, s);

RandQSORT(A, s + 2, q).

Randomized Quick Sort – Version 1

- **Analysis of RandQSORT**
- One execution of Step 2 needs **O(q-p) time.**
- The probability that the randomly chosen element is a central splitter is **½**.
- If **p be the probability of success of a random experiment**, and we continue the random experiment till we get success, the expected number of experiments we need to perform is **1/p**.
- The expected number of times Step 2 needs to be repeated to get a **central splitter** is 2 as the corresponding success probability is **½**.
- Thus, the expected time complexity for Step 2 is **O(n).**

Randomized Quick Sort – Version 1

- **Analysis of RandQSORT**
- The expected running time for the algorithm on a set **A**, excluding the time spent on recursive calls, is **O(A)**.
- Worst case size of each partition in **jth** level of recursion is **n. $(3/4)^j$**, so the expected time spend excluding recursive calls is **O(n. $(3/4)^j$)** for each partition.
- The number of partition of size **n. $(3/4)^j$** is **O($(3/4)^j$).**
- By linearity of expectations, the expected time for all partitions of size **n. $(3/4)^j$** is **O(n).**
- Number of levels of recursion **= $\log_{4/3} n$ = O(log n).**
- Thus, the expected running time is **O(n log n)**

Randomized Quick Sort – Version 2

- We randomized our algorithm by explicitly permuting the input.
- We could do so for quicksort also, but a different randomization technique, called **random sampling**, yields a simpler analysis.
- Instead of always using $A[r]$ or $A[0]$ as the pivot, we will select a randomly chosen element from the subarray $A[p..r]$.

Randomized Quick Sort – Version 2

- First exchanging element $\boldsymbol{A[r]}$ with an element chosen at random from $\boldsymbol{A[p..r]}$.
- By randomly sampling the range $\boldsymbol{p, \ldots, r}$, we ensure that the pivot element $\boldsymbol{x = A[r]}$ is equally likely to be any of the $\boldsymbol{r - p + 1}$ elements in the subarray.
- Because we randomly choose the pivot element, we expect the split of the input array to be reasonably well balanced on average.

Randomized Quick Sort – Version 2

- **RANDOMIZED-PARTITION.(A,p,r)**

  i = RANDOM.(p,r)

  exchange A[r] with A[i]

  RETURN PARTITION.(A,p,r)

- The new quicksort calls RANDOMIZED-PARTITION in place of PARTITION:

- **RANDOMIZED-QUICKSORT.(A,p,r)**

IF p < r

  q = RANDOMIZED-PARTITION.(A,p,r)

  RANDOMIZED-QUICKSORT.(A,p,q-1)

  RANDOMIZED-QUICKSORT.(A,q+1,r)

Randomized Quick Sort – Version 2

- **Analysis of Quick Sort**
- **Worst case analysis**
- A worst-case split at every level of recursion in quicksort produces a $\Theta(n^2)$ running time, which, intuitively, is the worst-case running time of the algorithm.
- Proof:
- $T(n) = MAX_{0 \le q \le n-1}\{T(q) + T(n-q-1)\} + \theta(n)$
- Where **q** ranges from **0** to **n-1** because the procedure PARTITION produces two subproblems with size n-1.

Randomized Quick Sort – Version 2

- **Worst case analysis**
- $\boldsymbol{T(n) = MAX_{0 \le q \le n-1}\{T(q) + T(n-q-1)\} + \theta(n)}$
- We guess that $T(n) \le cn^2$ for some constant c.
- So $\boldsymbol{T(n) = MAX_{0 \le q \le n-1}\{cq^2 + c(n-q-1)^2\} + \theta(n)}$
- $\boldsymbol{= c.MAX_{0 \le q \le n-1}\{q^2 + (n-q-1)^2\} + \theta(n)}$
- $\boldsymbol{\le cn^2 - c(2n-1) + \theta(n) \le cn^2}$
- Thus $T(n) = O(n^2)$.
- **When partition is unbalanced, $\boldsymbol{T(n) = \Omega(n^2)}$ is worst case running time.**

Randomized Quick Sort – Version 2

- **Expected running time**
- The expected running time of RANDOMIZED-QUICKSORT is O(n log n);
- if, in each level of recursion, the split induced by RANDOMIZED-PARTITION puts any constant fraction of the elements on one side of the partition, then the recursion tree has depth $\boldsymbol{\theta(\log n)}$ and $\boldsymbol{O(n)}$ work is performed at each level.
- Even if we add a few new levels with the **most unbalanced split** possible between these levels, the total time remains $\boldsymbol{O(n \log n)}$.

Randomized Quick Sort – Version 2

- **Expected running time**
- the expected running time of RANDOMIZED-QUICKSORT precisely by first understanding how the partitioning procedure operates and then using this understanding to derive an $O(n \log n)$; bound on the expected running time.

Randomized Quick Sort – Version 2

- **Indicator Random Variable**
- The **RANDOMIZED-PARTITION** procedure chooses each pivot randomly and independently.
- Consider an input to quicksort of the numbers 1 through 10 (in any order), and suppose that the first pivot element is 7.
- Then the first call to PARTITION separates the numbers into two sets: {1, 2, 3, 4, 5, 6} and {8, 9, 10}.
- In doing so, the pivot element 7 is compared to all other elements, but no number from the first set (e.g., 2) is or ever will be compared to any number from the second set (e.g., 9).

Randomized Quick Sort – Version 2

- **Indicator Random Variable**
- We assume that element values are distinct, once a pivot x is chosen with $Z_i < x < Z_j$, we know that Z_i and Z_j cannot be compared at any subsequent time.
- If, on the other hand, Z_i is chosen as a pivot before any other item in Z_{ij}, then Z_i will be compared to each item in Z_{ij} , except for itself.
- Similarly, if Z_j is chosen as a pivot before any other item in Z_{ij} , then Z_j will be compared to each item in Z_{ij} , except for itself.
- Thus, Z_i and Z_j are compared if and only if the first element to be chosen as a pivot from Z_{ij} is either Z_i or Z_j .

Randomized Quick Sort – Version 2

- **Indicator Random Variable**
- We now compute the probability that this event occurs.
- Prior to the point at which an element from Z_{ij} has been chosen as a pivot, the whole set Z_{ij} is together in the same partition.
- Therefore, any element of Z_{ij} is equally likely to be the first one chosen as a pivot.
- Because the set Z_{ij} has $j - i + 1$ elements, and because pivots are chosen randomly and independently, the probability that any given element is the first one chosen as a pivot is $1/(j - i + 1)$.

Randomized Quick Sort – Version 2

- **Indicator Random Variable**
- $P_r\{Z_i \text{ is compared to } Z_j\} = P_r\{Z_i \text{ or } Z_j \text{ is first pivot chosen from } Z_{ij}\}$
  $= P_r\{Z_i \text{ is first pivot chosen from } Z_{ij}\} + P_r\{Z_j \text{ is first pivot chosen from } Z_{ij}\}$

$$= \frac{1}{j-i+1} + \frac{1}{j-i+1} = \frac{2}{j-i+1}$$

- Combining we get that $E[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{2}{j-i+1}$

$$= \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{2}{k+1} = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{2}{k} = \sum_{i=1}^{n-1} O(\log n) = O(n \log n)$$

- The expected running time is $O(n \log n)$

HIRING PROBLEM

MODULE 6

Hiring problem

- Suppose that **you need to hire a new office assistant**. Your previous attempts at hiring have been unsuccessful, and you decide to use an employment agency.
- The employment agency will send you one candidate each day. You will interview that person and then decide to either hire that person or not. You must pay the employment agency a small fee to interview an applicant.
- To actually hire an applicant is more costly, however, **since you must fire your current office assistant and pay a large hiring fee to the employment agency**. You are committed to having, at all times, the best possible person for the job.

Hiring problem

- Therefore, you decide that, after interviewing each applicant, if that applicant is better qualified than the current office assistant, you will fire the current office assistant and hire the new applicant.
- You are willing to pay the resulting price of this strategy, **but you wish to estimate what that price will be.**
- The procedure HIRE-ASSISTANT, given below, expresses this strategy for hiring in pseudocode.
- It assumes that the **candidates for the office assistant job are numbered 1 through n**. The procedure assumes that you are able to, after interviewing candidate i, determine if candidate i is the best candidate you have seen so far.
- To initialize, the procedure creates a dummy candidate, numbered 0, who is less qualified than each of the other candidates.

Hiring problem

```
HIRE-ASSISTANT(n)
1  best ← 0     ®candidate 0 is a least-qualified dummy candidate
2  for i ← 1 to n
3       do interview candidate i
4          if candidate i is better than candidate best
5             then best ← i
6                  hire candidate i
```

Hiring problem

- We are not concerned with the running time of HIRE-ASSISTANT, but instead with the **cost incurred by interviewing and hiring.**
- On the surface, analyzing the cost of this algorithm may seem very different from analyzing the running time of, say, merge sort.
- The analytical techniques used, however, are identical whether we are analyzing cost or running time.
- In either case, **we are counting the number of times certain basic operations are executed**.

Hiring problem

- Interviewing has a low cost, say c_i, whereas hiring is expensive, costing c_h.
- Let m be the number of people hired. Then the total cost associated with this algorithm is $O(nc_i\, mc_h)$.
- No matter how many people we hire, we always interview n candidates and thus always incur the cost nc_i associated with interviewing.
- We therefore concentrate on analyzing mc_h, the hiring cost. This quantity varies with each run of the algorithm.

Hiring problem

- It is often the case that we need to find the maximum or minimum value in a sequence by examining each element of the sequence and maintaining a current "winner."
- The hiring problem models how often we update our notion of which element is currently winning.

Hiring problem – Worst Case Analysis

- In the worst case, we actually hire every candidate that we interview. This situation occurs if the candidates come in **increasing order of quality**, in which case we hire **n** times, for a total hiring cost of $\mathbf{O(nc_h)}$.
- It might be reasonable to expect, however, that the candidates do not always come in increasing order of quality.
- In fact, **we have no idea about the order in which they arrive, nor do we have any control over this order**. Therefore, it is natural to ask what we expect to happen in a typical or average case.

Hiring problem – Probabilistic Analysis

- **Probabilistic analysis** is the use of probability in the analysis of problems.
- Most commonly, we **use probabilistic analysis to analyze the running time of an algorithm.**
- Sometimes, we use it to analyze other quantities, such as the hiring cost in procedure HIRE-ASSISTANT.
- In order to perform a probabilistic analysis, we must use knowledge of, or **make assumptions about, the distribution of the inputs**.
- Then we analyze our algorithm, computing an expected running time.
- The **expectation is taken over the distribution of the possible inputs**.
- Thus we are, in effect, averaging the running time over all possible inputs.

Hiring problem – Probabilistic Analysis

- For some problems, it is reasonable to assume **something about the set of all possible inputs**, and we **can use probabilistic analysis** as a technique for designing an efficient algorithm and as a means for gaining insight into a problem.
- For other problems, **we cannot describe a reasonable input distribution**, and in these cases we cannot use probabilistic analysis.

Hiring problem – Probabilistic Analysis

- For the hiring problem, **we can assume that the applicants come in a random order**.
- What does that mean for this problem?
- We assume that **we can compare any two candidates and decide which one is better qualified;** that is, there is a total order on the candidates.

Hiring problem – Probabilistic Analysis

- We can therefore rank each candidate with a unique number from **1** through ***n***, using ***rank(i)*** to denote the rank of applicant **i**, and adopt the convention that a higher rank corresponds to a better qualified applicant.
- The ordered list ***<rank(1), rank(2), …, rank(n)>*** is a permutation of the list ***<1, 2, …, n>.*** Saying that the applicants come in a random order is equivalent to saying that this list of ranks is equally likely to be any one of the ***n!*** permutations of the numbers ***1*** through ***n***.
- Alternatively, we say that the ranks form a **uniform random permutation**; that is, each of the possible n! permutations appears with equal probability.

Hiring problem – Randomized Algorithms

- In order to use probabilistic analysis, we need to know something about the distribution of the inputs.
- **In many cases, we know very little about the input distribution.**
- Even if we do know something about the distribution, we may not be able to model this knowledge computationally.
- Yet we often can use **probability and randomness** as a tool for algorithm design and analysis, by making the behavior of part of the algorithm random.

Hiring problem – Randomized Algorithms

- In the hiring problem, it may seem as **if the candidates are being presented to us in a random order**, but we have no way of knowing whether or not they really are.
- Thus, in order to develop a randomized algorithm for the hiring problem, we **must have greater control over the order in which we interview the candidates**.
- We will, therefore, change the model slightly. We say that the employment agency has **n** candidates, and they send us a list of the candidates in advance.

Hiring problem – Randomized Algorithms

- On each day, we choose, randomly, which candidate to interview.
- Although we know nothing about the candidates (besides their names), we have made a significant change.
- Instead of **relying on a guess that the candidates come to us in a random order, we have instead gained control of the process and enforced a random order**.

Hiring problem – Randomized Algorithms

- An algorithm randomized if its behavior is determined not only by its input but also by values produced by a ***random-number generator***.
- We shall assume that we have at our disposal a random-number generator **RANDOM**.
- A call to **RANDOM (a, b)** returns an integer between a and b, inclusive, with each such integer being equally likely.

Hiring problem – Randomized Algorithms

- For Example, **RANDOM(0,1)** produces 0 with probability ½, and it produce 1 with probability ½.
- A call to **RANDOM (3,7)** returns either 3,4,5,6,or 7 each with probability 1/5.
- Each integer returned by **RANDOM** is independent of the integers returned on previous calls.
- **In practice, most programming environment offers a pseudorandom-number generator: a deterministic algorithm returning numbers that "look" statistically random.**

Hiring problem – Randomized Algorithms

- **Analyze of Randomized Algorithm**:
- Expectation of the running time over the distribution of values returned by random number generator.
- Algorithms are distinguished by random inputs by referring to the running time – **expected running time.**
- Average-case running time : **probability distribution is over the inputs** to the algorithm

Indicator Random Variables

- In order to analyze many algorithms, including the hiring problem, we use indicator random variables.
- **Indicator random Variables** provide a convenient method for converting between probabilities and expectations.

Indicator Random Variables

- Suppose we are given a sample space $\boldsymbol{S}$ and an event $\boldsymbol{A}$.
- Then the indicator random variable $\boldsymbol{I\{A\}}$ associated with event **A** is defined as
- $$\boldsymbol{I\{A\} = \begin{cases} 1 & 1\ if\ A\ occurs \\ 0 & if\ A\ does\ not\ occurs \end{cases}}$$

Indicator Random Variables

- Example:
- Our sample space is $S = \{H, T\}$, with $P_r\{H\} = P_r\{T\} = ½$.
- We define an indicator random variable X_H, associated with the coin coming up heads, which is the event H.
- This variable counts the number of heads obtained in this flip, and it is **1** if the coin comes up heads and **0** otherwise.
- $X_H = I\{H\} = \begin{cases} 1 & if\ H\ occurs \\ 0 & if\ T\ occurs \end{cases}$

Indicator Random Variables

- Example:
- The expected number of heads obtained in one flip of the coin is simply the expected value of our indicator variable $\boldsymbol{X_H}$:
- $\boldsymbol{E[X_H] = E[I\{H\}] = 1.P_r\{H\} + 0.P_r\{T\} = 1.\left(\frac{1}{2}\right) + 0.\left(\frac{1}{2}\right) = \frac{1}{2}}$
- The expected number of Heads obtained by one flip of fair coin is **½**.

Indicator Random Variables

- Lemma: Given a sample space $\boldsymbol{S}$ ad an event $\boldsymbol{A}$ in the sample space $\boldsymbol{S}$, let $\boldsymbol{X_A = I\{A\}.}$ Then $\boldsymbol{E[XA] = P_r\{A\}.}$
- Proof: By the definition,
- $\boldsymbol{E[X_A] = E[I\{A\}] = 1.P_r\{A\} + 0.P_r\{\overline{A}\} = P_r\{A\}}$
- Where $\overline{\boldsymbol{A}}$ denotes $\boldsymbol{S - A}$, the complement of $\boldsymbol{A}$.

Indicator Random Variables

- Let us consider X_i be the indicator random variable associated with **ith** flip comes up heads: $X_i = I\{\textit{the ith flip results in the event } H\}$.
- Let X be the random variable denoting the total number of heads in the **n** coin flips, so
- $X = \sum_{i=1}^{n} X_i$
- The expectation of the both sides of the above is
- $E[X] = E[\sum_{i=1}^{n} X_i]$
- To compute the expectation of the sum: it equals the sum of the expectation of the n random variables.

Indicator Random Variables

- To compute the expectation of the sum: it equals the sum of the expectation of the n random variables.
- $\boldsymbol{E[X] = E[\sum_{i=1}^{n} X_i] = \sum_{i=1}^{n} E[X_i] = \sum_{i=1}^{n} 1/2 = n/2}$

Hiring Problem -Indicator Random Variables

- Let **X** be the random variable whose value equals the number of times we hire a new office assistants.
- By the definition $E[X] = \sum_{x=1}^{n} xP_r\{X = x\}$.
- To simplify the calculation, let X_i be the indicator random variable associated with the event in which the **ith** candidate is hired
- $X_i = I\{candidate\ i\ is\ hired\} = \begin{cases} 1 & if\ candidate\ i\ is\ hired \\ 0 & if\ candidate\ i\ is\ not\ hired' \end{cases}$

  and $X = X_1 + X_2 + \cdots = X_n$.

Hiring Problem -Indicator Random Variables

- Lemma: $E[X_i] = Pr\{candidate\ i\ is\ hired\}$
- Using **HIRE-ASSISTANT**, Candidate i has a probability of $1/i$ of being better qualified than candidates 1 through $i-1$ and thus a probability of $1/i$ of being hired.
- We conclude that $E[X_i] = 1/i$, We computer $E[X]$:
- $E[X] = E[\sum_{i=1}^{n} X_i] = \sum_{i=1}^{n} E[X_i] = \sum_{i=1}^{n} 1/i = log.n + O(1)$
- We interview **n** people, we actually hire only approximately **log.n** of them, on average $O(c_h.log.n)$

Hiring Problem –Randomized Algorithm

- We randomized the hiring problem by **knowing a distribution on the inputs** that can help us to analyze the average-case distribution of an algorithm.
- Previously we randomly permutate the candidate in order to enforce the property that every permutation is equally likely.
- But now **we expect that this to be in case of any input**, **rather than for inputs drawn form a particular distribution.**

Hiring Problem –Randomized Algorithm

- Note that the algorithm here is deterministic; for any particular input, the number of times a new office assistant is hired is always the same.
- Furthermore, the number of times we hire a new office assistant differs for different inputs, and **it depends on the ranks of the various candidates.**
- Since this number depends only on the ranks of the candidates, we can represent a particular input by listing, in order, the ranks of the candidates, i.e**., {rank(1), rank(2),…, rank(n)}.**

Hiring Problem –Randomized Algorithm

- Given the rank list $\boldsymbol{A_1 = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}}$, a new office assistant is always hired 10 times, sinch each successive candidate is better than the previous one.
- Given the rank list $\boldsymbol{A_2 = \{10, 9, 8, 7, 6, 5, 4, 3, 2, 1\}}$, a new office assistant is always hired in first iteration.
- Given the rank list $\boldsymbol{A_3 = \{5, 2, 1, 8, 4, 7, 10, 9, 3, 6\}}$, a new office assistant is hired three times, upon interviewing the candidates with rank 5,8, and 10.
- We see that there are expensive inputs such as $\boldsymbol{A_1}$, inexpensive inputs such as $\boldsymbol{A_2}$, and moderately expensive inputs such as $\boldsymbol{A_3}$.

Hiring Problem –Randomized Algorithm

- In this case, we randomize in the algorithm, not in the input distribution.
- Given a particular input, say $\boldsymbol{A_3}$ above, we cannot say how many times the maximum is updated, because this quantity differs with each run of the algorithm.
- The first time we run the algorithm on $\boldsymbol{A_3}$, it may produce the permutation $\boldsymbol{A_1}$ and **perform 10 updates**; but the second time we run the algorithm, we may produce the permutation $\boldsymbol{A_2}$ and **perform only one update**.
- The third time we run it, we may perform some other number of updates.

Hiring Problem –Randomized Algorithm

- Each time we run the algorithm, the execution depends on the random choices made and is likely to differ from the previous execution of the algorithm.
- For this algorithm and many other randomized algorithms, ***no particular input elicits its worst-case behavior***. Even your worst enemy cannot produce a bad input array, since the random permutation makes the input order irrelevant.
- The randomized algorithm performs badly only if the random-number generator produces an **"unlucky"** permutation.

Hiring Problem –Randomized Algorithm

- So we change algorithm to code to randomly permute the array.

RANDOMIZED-HIRE-ASSISTANT(n)

```
1 randomly permute the list of candidates
2 best = 0 // candidate 0 is a least-qualified dummy candidate
3 for i = 1 to n
4         interview candidate i
5         if candidate i is better than candidate best
6                 best = i
7                 hire candidate i
```

Hiring Problem –Randomized Algorithm

◦ So we change algorithm to code to randomly permute the array.

PERMUTE-BY-SORTING(A)

1 n = A.length

2 let P [1..n] be a new array

3 for i = 1 to n

4 P[i] = RANDOM(1, n^3)

5 sort A, using P as sort keys

Hiring Problem –Randomized Algorithm

- The comparison sort takes $\boldsymbol{\Omega(n \log n)}$ time.
- We can achieve this lower bound, if we have used merge sort which takes $\boldsymbol{\Theta(n \log n)}$ time.
- After sorting, if **P[i]** is the **j**th smallest priority, then **A[i]** lies in position **j** of the output.
- In this manner we obtain a permutation. It remains to prove that the procedure produces a **uniform random permutation**, that is, that the procedure is equally likely to produce every permutation of the number **1** through **n**.

GLOBAL MINCUT

MUDULE 6

Global Mincut Problem

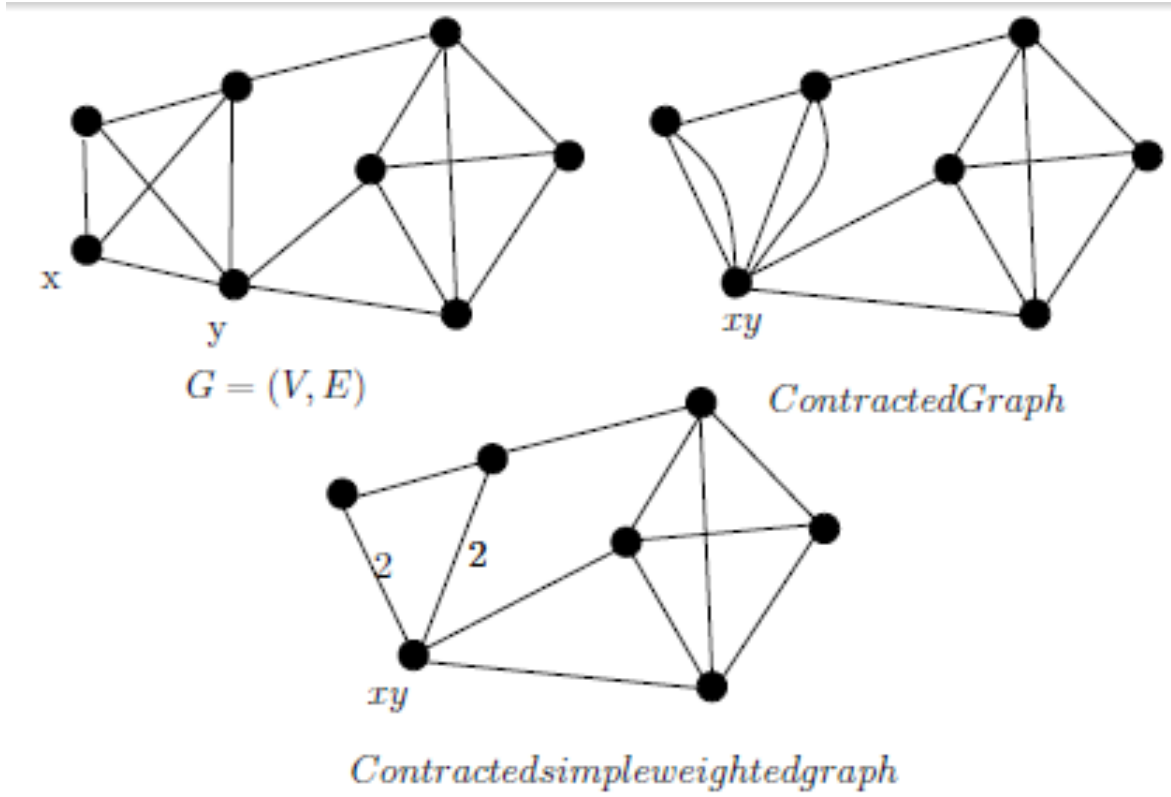
- **Problem**:
- Given an undirected graph **G=(V,E)**, we define a **cut** of **G** to be partition of **V** into two non=empty sets A and B.
- In Network Flows, we worked with **S-T** cut, given a directed graph **G=(V,E)** with source **S** and sink **T,** an **S-T** cut was defined to be a partition of **V** into set **A** and **B** such that $S \in A$ and $T \in B$
- A **global minimum cut** is a cut of minimum size. The **global** refers to connote that any cut of the graph is allowed; there is no source or sink.
- **Global min-cut** is a natural "robustness" parameter, it is the smallest number of edges whose deletion disconnects the graph.

Global Mincut Problem

- Applications:
- Clustering and partitioning items
- Network reliability, network design, circuit design etc.

Global Mincut Problem

- **A Simple Randomized Algorithm:**
- **Contraction of an Edge**
- Contraction of an edge **e = (x,y)** implies merging the two vertices **x,y ∈ V** into a single vertex, and remove the self loop. The contracted graph is denoted by **G/xy.**



Global Mincut Problem

- **Contraction of Edge:**
- Result – 1 : As long as G/xy has at least one edge,
  - The size of the minimum cut in the (weighted) graph G/xy is at least as large as the size of the minimum cut in G .
- Result – 2 : Let $e_1, e_2, \ldots, en$ be a sequence of edges in G, such that
  - none of them is in the minimum cut of G, and
  - $G' = G/\{e_1, e_2, \ldots, en\}$ is a single multiedge.
- Then this multiedge corresponds to the minimum cut in G.

Global Mincut Problem

- **Algorithm MINCUT(G):**

$G_0 \leftarrow G$; $i=0$

WHILE ***G_i*** has more than two vertices DO

Pick randomly an edge ***e_i*** from the edges in ***G_i***

$G_{i+1} \leftarrow G_i / e_i$

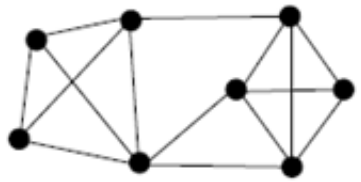
$i \leftarrow i+1$

(S, V-S) is the cut in the original graph corresponding to the single edge in ***G_i***

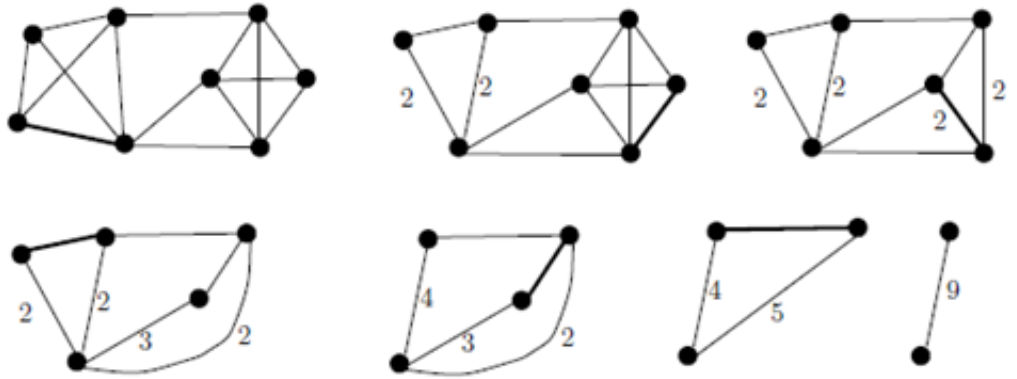
- Time complexity is $O(n^2)$

Global Mincut Problem

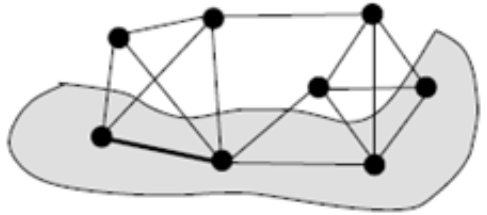
The given graph:



Stages of Contraction:



The corresponding output:



Global Mincut Problem

- **Quality Analysis**
- Lower bounding ***|E|***
  - If a graph ***G = (V, E)*** has a minimum cut ***F*** of size ***k***, and it has ***n*** vertices, then ***|E| ≥ (kn)/2***
  - Proof: If any node ***v*** has degree less than ***k***, then the ***cut({v}, V-{v})*** will have size less than ***k***.
  - This contradicts the fact that ***(A; B)*** is a global min-cut.
  - Thus, every node in ***G*** has degree at least ***k***. So, ***|E| ≥ ½ kn***
- So the probability that an edge in ***F*** is contracted is at most $\frac{k}{(kn)/2} = \frac{2}{n}$

Global Mincut Problem

- **Quality Analysis**
- If we pick a random edge e from the graph ***G,*** then the probability of e belonging in the **mincut** is at most ***2/n***
- **Continuing Contraction**
- After ***i*** iterations, there are ***n - i*** supernodes in the current graph ***G'*** and suppose no edge in the cut ***F*** has been contracted.
- Every cut of ***G'*** is a cut of ***G***. So, there are at least k edges incident on every supernode of ***G'.*** Thus, ***G'*** has at least ***1/2 k(n - i)*** edges.
- So, the probability that an edge in ***F*** is contracted in iteration ***i + 1*** is at most ***k; 1, 2,…,*** $\frac{k}{1/2k(n-i)} = \frac{2}{n-i}$.

Global Mincut Problem

- **Lower Bounding the Intersection of Events**
- We want to lower bound $Pr[\eta 1 \cap \cdots \cap \eta n - 2]$. We use earliest result

$$Pr[\cap_{i=1}^{n} \eta_i] = Pr[\eta_1] \cdot Pr[\eta_2 \mid \eta_1] \cdot Pr[\eta_3 \mid \eta_1 \cap \eta_2] \cdots Pr[\eta_n \mid \eta_1 \cap \ldots \cap \eta_{n-1}].$$

So, we have $Pr[\eta_1] \cdot Pr[\eta_1 \mid \eta_2] \cdots Pr[\eta_{n-2} \mid \eta_1 \cap \eta_2 \cdots \cap \eta_{n-3}]$

$$\geq \left(1 - \frac{2}{n}\right)\left(1 - \frac{2}{n-1}\right) \cdots \left(1 - \frac{2}{n-i}\right) \cdots \left(1 - \frac{2}{3}\right)$$

$$= \binom{n}{2}^{-1}$$

Global Mincut Problem

- **Bounding the Error Probability**
- We know that a single run of the contraction algorithm fails to find a global min-cut with probability at most $1 - \frac{1}{\frac{n}{2}} \approx 1$.
- We can amplify our success probability by repeatedly running the algorithm with independent random choices and taking the best cut.
- If we run the algorithm $\binom{n}{2}$ times, then the probability that we fail to find a global min-cut in any run is at most $\left(1 - \frac{1}{\binom{n}{2}}\right)^{\binom{n}{2}} \leq \frac{1}{e}$
- By spending $O(n^4)$ time, we can reduce the failure probability from $1 - \frac{2}{n^2}$ to a reasonably small constant value 1/e.

Global Mincut Problem

- **Probability in counting**
- Consider $\mathbf{C_n}$ a cycle on n nodes. How many global minimum cut are possible?
- Cut out any two edges to have **(n/2)** such cuts.
- Let there be **r** such cuts $\mathbf{C_1, \ldots, C_r}$.
- Let $\boldsymbol{\varepsilon = U_{i=1}^{r} \varepsilon_i}$ is the even that the algorithm returns any global minimum cut.
- Basically $\mathbf{Pr[\varepsilon_i] \geq 1/(n/2)}$, each pair of events $\boldsymbol{\varepsilon_i}$ and $\boldsymbol{\varepsilon_j}$ are disjoint since only one cut is returned by any run of the algorithm
- By union bound, we have $\mathbf{Pr[\varepsilon] = Pr[U_{i=1}^{r} \varepsilon_i] = \sum_{i=1}^{r} Pr[\varepsilon_i] \geq \frac{r}{\binom{n}{2}}}$. **So** $\mathbf{Pr[\varepsilon] \leq 1}$. **So** $\mathbf{r \leq \binom{n}{2}}$

Randomized Algorithm

- Employing randomness **leads to improved simplicity and improved efficiency in solving the problem**.
- It assumes the availability of a **perfect source of independent and unbiased random bits.**
- Access to truly unbiased and independent sequence of random bits is expensive. So, it should be considered as an expensive resource like time and space.
- There are ways to reduce the randomness from several algorithms while maintaining the efficiency nearly the same.

MODULE 6

BCSE206L – DESIGN AND ANALYSIS OF ALGORITHMS